Advanced CPU Architecture and Hardware Accelerators - Lab 1

Maayan Sarig 206511776

Ido Mayerowicz 207180571

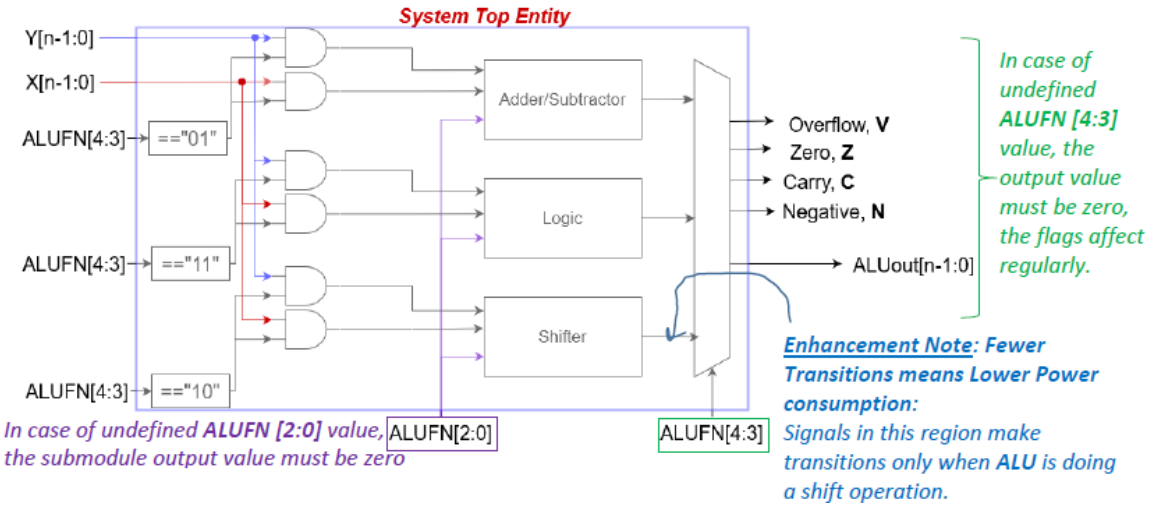
Date: 04.03.2025

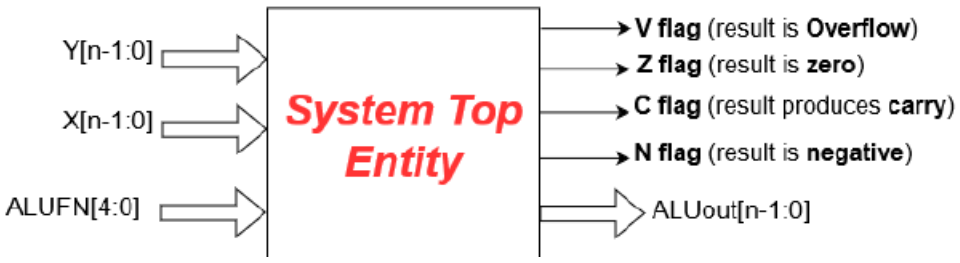
מבוא

מטרת מעבדה 1 הינה לרכוש כלים בסיסיים בשפת VHDL. במעבדה זו תכננו מודול (Top) המכיל שלושה תתי-מודולים:

- מודול לחיבור/חיסור של שני מספרים (Adder/Subtractor).
- מודול לביצוע הזזות בתצורת Barrel-Shifter, הזזה של מספר Y ימינה/שמאלה X הזזות (Shifter).
- מודול לביצוע פעולות לוגיות בין שני מספרים (Logic).

ארכיטקטורת המערכת הכוללת נראית כך:





כעת ננתח כל מודול בפני עצמו על מנת להסביר את אופן העבודה של המערכת כולה.

לאחר הסבר על כל מודול, אופן פעולה ולוגיקה פנימית נסביר את אופן ביצוע הסימולציה לבדיקת נכונות תפקוד המערכת.

חשוב לציין כי ביצענו סימולציה על כל תת מודול בנפרד וזאת על מנת להוריד מרמת הקושי בשלב ה debug של המערכת כולה, כל סימולציה מורכבת מסדרת קלטים אותם בדקנו בצורה ידנית.

תיאור המערכת

המערכת מקבלת ככניסות שני אותות כניסה Y,X וקו בקרה ALUFN, קו הבקרה מגדיר את תת המודול הנדרש לפעולה וכן את הפעולה המבוקשת באופן הבא:

Function Kind	Decimal value	ALUFN	Operation	Note
Arithmetic	8	01000	Res=Y+X	
	9	01001	Res=Y-X	Used also for comparison operation
	10	01010	Res=neg(X)	
	11	01011	Res=Y+1	Increment of Y in one
	12	01100	Res=Y-1	Decrement of Y in one
Shift	16	10000	Res=SHL Y,X(k-1 to 0)	Shift Left Y of q≜X(k-1 ..0) times Res=Y(n-1-q…0)#(q@0) $When\ k = log_2 n$
	17	10001	Res=SHR Y,X(k-1 to 0)	Shift Right Y of q≜X(k-1 ..0) times Res=(q@0)#Y(n-1...q) $When\ k = log_2 n$
Boolean	24	11000	Res=not(Y)	
	25	11001	Res=Y or X	
	26	11010	Res=Y and X	
	27	11011	Res=Y xor X	
	28	11100	Res=Y nor X	
	29	11101	Res=Y nand X	
	30	11111	Res=Y xnor X	

תת המודול הרצוי

הפעולה הרצויה

מוצא המערכת הינו התוצאה של הפעולה המתמטית המבוקשת, כמו כן למערכת קיימת האפשרות להרים 4 דגלים:

- N (Negative) – עבור מוצא שלילי.
- C (Carry) – עבור פעולת הזזה/חיבור/חיסור, עבור פעולות לוגיות תמיד 0.
- Z (Zero) – כאשר המוצא הינו 0.
- V (Overflow) – כאשר מוצא המערכת לא ניתן לייצוג ע"י מספר הביטים המוקצה.

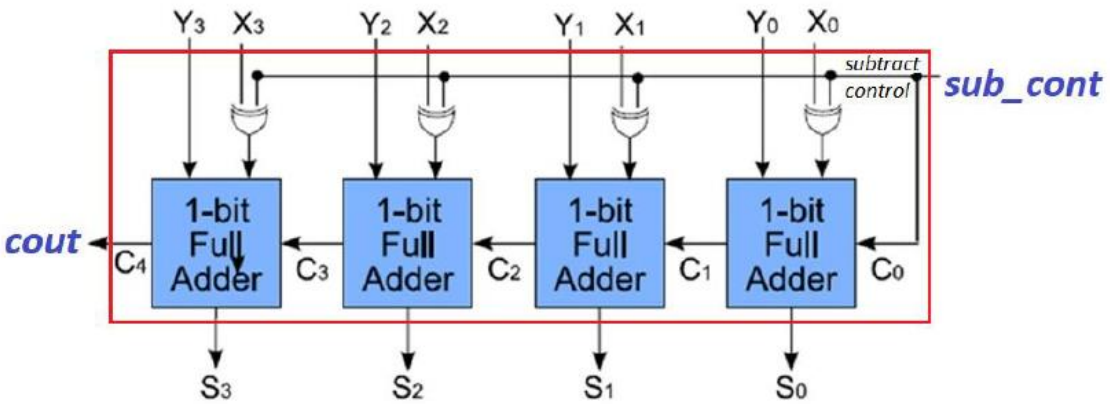
בהמשך נצלול לתוך כל תת מודול ונסביר את אופן הפעולה הפנימית של כל אחד.

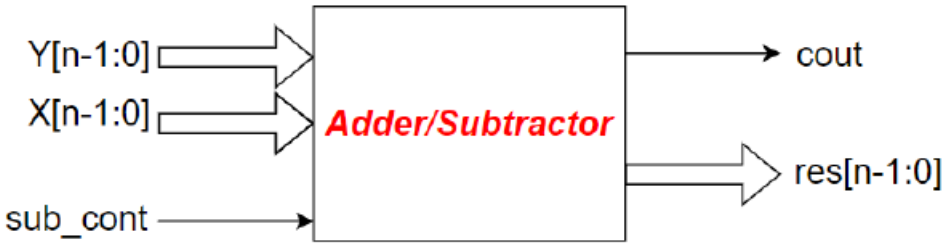
מודול Adder/Subtractor

המודול מבצע פעולת חיבור חיסור בין 2 כניסת Y,X ע"פ קו הבקרה ALUFN באופן הבא:

- פעולת חיבור מתבצעת ע"י חיבור פשוט של האיברים.
- פעולת חיסור מתבצעת ע"י תכונת משלים ל2 (XOR עם 1 ו- carry=1).

המודול עצמו הינו חיבור טורי של מחברים מחדרים, כפי שניתן לראות בשרטוט הבא:





בשרטוט ניתן להבחין באופן הפעולה של המודול עבור בוררות של חיבור או חיסור בין הביטים:

- כאשר בקו הבקרה sub_cont יש '0' אזי נכנס לFA 'X'.
- כאשר בקו הבקרה sub_cont יש '1' אזי נכנס לFA 1 xor 'X'.

ע"י שרשור של FA נקבל את וקטור התוצאה ובהתאם גם את הנשא Cout.

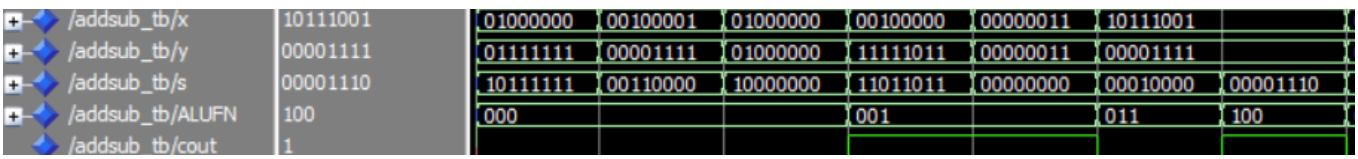
סימולציה Adder/Subtractor

בסימולציה בדקנו את העילות התקינה של המודול, על מנת לעשות זו בדקנו את מצבי הפעולה הבאים:

- חיבור ללא נשא Cout
- חיבור עם נשא Cout
- חיסור
- Negative

מצורפת תמונה של הרצת הסימולציה בשתי תצורות ההצגה:

ps	delta	/addsub_tb/x	/addsub_tb/y	/addsub_tb/s	/addsub_tb/ALUFN	/addsub_tb/cout
0	+4	01000000	01111111	10111111	000	0
50000	+7	00100001	00001111	00110000	000	0
100000	+4	01000000	01000000	10000000	000	0
150000	+5	00100000	11111011	11011011	001	1
200000	+3	00000011	00000011	00000000	001	1
250000	+4	10111001	00001111	00010000	011	0
300000	+4	10111001	00001111	00001110	100	1



מודול Shifter

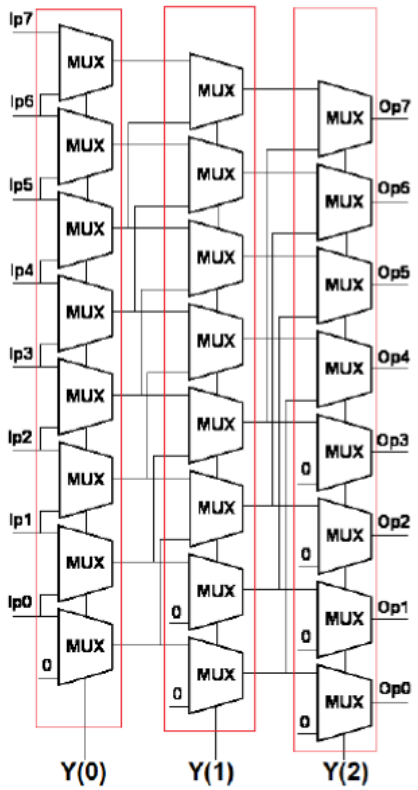
מודול זה מבצע הזזה של הקלט Y כמספר הפעמים שהקלט X מורה זאת, קו הבקרה מורה האם ההזזות מתבצעות ימינה או שמאלה. המימוש החומרתי הינו של Barrel-Shifter כפי שלמדנו בקורס מבוא למחשבים.

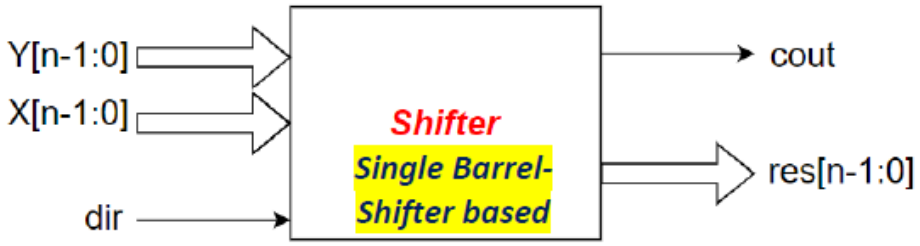
על מנת לכתוב קוד פשוט למימוש חישבנו את כל ההזזות הקיימות עבור אותה כניסה, כך שכל וקטור שורה הינו ההזזה של הוקטור שמעליו ימינה/שמאלה בהתאם לקו הבקרה, כאשר השורה העליונה ביותר של המטריצה מוגדרת להיות וקטור הקלט Y.

עבור הזזה ימינה המטריצה נבנית באותו אופן אך בעת ההשמה עצמה של הוקטור לשורה הבאה נבצע היפוך לוקטור המקורי וכמובן הזזה שמאלה וכך נקבל את וקטור השורה השקול.

לבסוף נבחר את השורה הרלוונטית המייצגת את הוקטור המקורי לאחר מספר ההזזות הרצויות, מספר השורה תואם את ההצגה הדסימלית של מספר ההזזות שמורה הקלט X.

המימוש החומרתי של מודול זה:





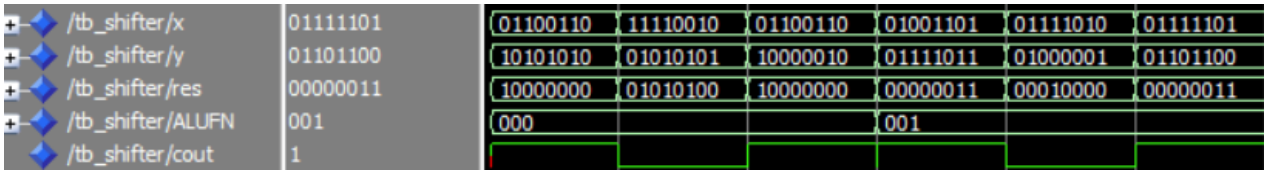
סימולציית Shifter

בסימולציה בדקנו את העילות התקינה של המודול, על מנת לעשות זו בדקנו את מצבי הפעולה הבאים:

- הזזה ימינה
- הזזה שמאלה

מצורפת תמונה של הרצת הסימולציה בשתי תצורות ההצגה:

ps	delta	/tb_shifter/x	/tb_shifter/y	/tb_shifter/ALUFN	/tb_shifter/cout
0	+9	01100110	10101010	10000000	000	1
50000	+5	11110010	01010101	01010100	000	0
100000	+9	01100110	10000010	10000000	000	1
150000	+8	01001101	01111011	00000011	001	1
200000	+5	01111010	01000001	00010000	001	0
250000	+8	01111101	01101100	00000011	001	1

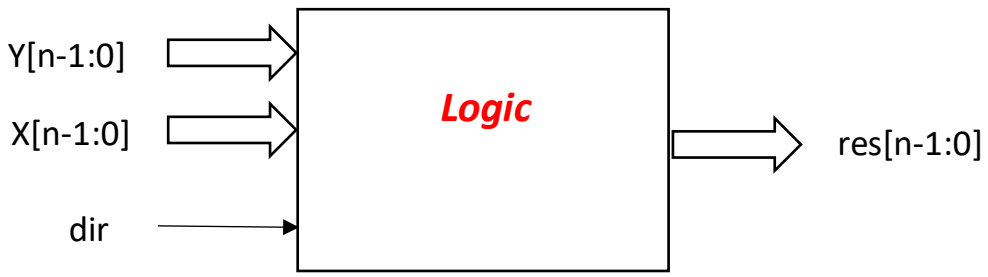


מודול Logic

מודול זה מבצע פעולות לוגיות של הקלטים Y,X בהתאם לקו הבקרה.

המודול בעל הפונקציונליות לבצע את הפעולות הלוגיות הבאות: not, or, and, xor, nor, nand, xnor.

חשוב לציין כי כלל הפעולות הלוגיות שכאן מתבצעות bit by bit ולכן בקוד עצמו בכל פעם בדקנו את שני הביטים הנמצאים באותו אינדקס עבור שני האותות (הוקטורים המייצגים את הקלטים).



סימולציית Logic

בסימולציה בדקנו את העילות התקינה של המודול, על מנת לעשות זו בדקנו את מצבי הפעולה הבאים:

- ביצוע פעולות or, not, and, xor, nor, nand, xnor

מצורפת תמונה של הרצת הסימולציה בשתי תצורות ההצגה:

ps delta	/tb_logic/x	/tb_logic/y	/tb_logic/ALUFN	/tb_logic/res
0 +2	11001100	11001100	000	00110011
50000 +2	00011110	00100101	001	00111111
100000 +2	00011100	10101010	010	00001000
150000 +2	10101010	10111010	011	00010000
200000 +2	11011001	10111010	100	00000100
250000 +2	00000111	00100101	101	11111010
300000 +2	11000001	10111010	110	00000000
350000 +2	11001110	00100101	111	00010100

/tb_logic/x	00011110	11001100	00011110	00011100	10101010	11011001	00000111	11000001	11001110
/tb_logic/y	00100101	11001100	00100101	10101010	10111010		00100101	10111010	00100101
/tb_logic/ALUFN	001	000	001	010	011	100	101	110	111
/tb_logic/res	00111111	00110011	00111111	00001000	00010000	00000100	11111010	00000000	00010100

מודול Top

מודול זה הינו מודול המעטפת של המערכת, בהתאם לקו ה ALUFN המודול מעביר את הכניסות לתת המודול המתאים ובהתאם גם אחראי למוצא תת המודל והרמת הדגלים המתאימים.

מכיוון והמערכת הינה מערכת ספרתית אזי שפעולה מתמטית אחת מתבצעת באותו הזמן כלומר רק אחד מתתי המודולים מוציא פלט נכון, אזי נגדיר מצב של high z כך שכל עוד המודול הנ"ל אינו בפעולה המוצא שלו יהיה high z ואילו המודול שבשימוש יוציא את הפלט הנכון אותו מעביר הלאה.

לאחר מעבר בתת המודול הרלוונטי יצא '1' או '0' בקו הבקרה Cout. הדלקת הדגלים מתבצעת בTop לכניסות ולמוצא, הדלקת האותות והתנאים המתאימים מופיעים בטבלה הבאה:

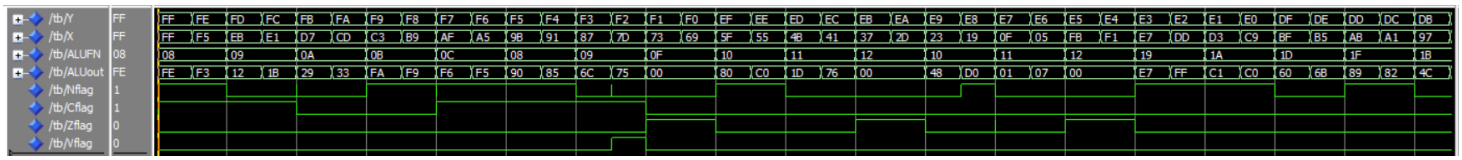
Status Bit	Description
V	Overflow bit. This bit is set when the result of an arithmetic operation overflows the signed-variable range. **1.** In case of ADD operation: V=1: • Positive + Positive = Negative • Negative + Negative = Positive otherwise, V=0. **2.** In case of SUB operation: V=1: • Positive - Negative = Negative • Negative - Positive = Positive otherwise, V=0. **Note:** It is mandatory to first manually write an optimized Boolean equation for V flag using above terms before its implementation.
Z	Zero bit. This bit is set when the result is 0 and cleared when the result is not 0.
C	Carry bit. This bit is set when the result produced a carry and cleared when no carry occurred.
N	Negative bit. This bit is set when the result is negative and cleared when the result is not negative.

סימולציית Top

סימולציית ה Top בודקת גם את תתי המודולים הרי שהיא משתמשת בהם לביצוע הפעולה המבוקשת, בהרצת הסימוליציה השתמשנו ב tb_ref1.vhd ובדקנו כי הערכים במוצא והדגלים יצאו בהתאם לטבלה שניצנה לנו עבור הפלטים הנכונים.

מצורפת תמונה של הרצת הסימולציה בשתי תצורות ההצגה:

ps	delta	/tb/Y	/tb/X	/tb/ALUout	/tb/ALUFN	/tb/Nflag	/tb/Cflag	/tb/Zflag	/tb/Vflag
150000	+8	11111100	11100001	01001	00011011	0	1	0	0
200000	+8	11111011	11010111	01010	00101001	0	0	0	0
250000	+6	11111010	11001101	01010	00110011	0	0	0	0
300000	+7	11111001	11000011	01011	11111010	1	0	0	0
350000	+7	11111000	10111001	01011	11111001	1	0	0	0
400000	+8	11110111	10101111	01100	11110110	1	1	0	0
450000	+7	11110110	10100101	01100	11110101	1	1	0	0
500000	+10	11110101	10011011	01000	10010000	1	1	0	0
550000	+8	11110100	10010001	01000	10000101	1	1	0	0
600000	+8	11110011	10000111	01001	01101100	0	1	0	0
650000	+9	11110010	01111101	01001	01110101	0	1	0	1
700000	+7	11110001	01110011	01111	00000000	0	0	1	0
750000	+1	11110000	01101001	01111	00000000	0	0	1	0
800000	+13	11101111	01011111	10000	10000000	1	0	0	0
850000	+11	11101110	01010101	10000	11000000	1	0	0	0
900000	+9	11101101	01001011	10001	00011101	0	0	0	0
950000	+5	11101100	01000001	10001	01110110	0	0	0	0
1000000	+5	11101011	00110111	10010	00000000	0	0	1	0
1050000	+1	11101010	00101101	10010	00000000	0	0	1	0
1100000	+9	11101001	00100011	10000	01001000	0	0	0	0
1150000	+7	11101000	00011001	10000	11010000	1	0	0	0
1200000	+13	11100111	00001111	10001	00000001	0	0	0	0
1250000	+5	11100110	00000101	10001	00000111	0	0	0	0
1300000	+5	11100101	11111011	10010	00000000	0	0	1	0
1350000	+1	11100100	11110001	10010	00000000	0	0	1	0
1400000	+5	11100011	11100111	11001	11100111	1	0	0	0
1450000	+5	11100010	11011101	11001	11111111	1	0	0	0
1500000	+5	11100001	11010011	11010	11000001	1	0	0	0
1550000	+5	11100000	11001001	11010	11000000	1	0	0	0
1600000	+5	11011111	10111111	11101	01100000	0	0	0	0
1650000	+5	11011110	10110101	11101	01101011	0	0	0	0
1700000	+5	11011101	10101011	11111	10001001	1	0	0	0
1750000	+5	11011100	10100001	11111	10000010	1	0	0	0
1800000	+5	11011011	10010111	11011	01001100	0	0	0	0
1850000	+5	11011010	10001101	11011	01010111	0	0	0	0
1900000	+3	11011001	10000011	00100	00000000	0	0	1	0
1950000	+1	11011000	01111001	00100	00000000	0	0	1	0
2000000	+1	11010111	01101111	00100	00000000	0	0	1	0
2050000	+1	11010110	01100101	00100	00000000	0	0	1	0


/tb/Y FF FF FE FD FC FB FA F9 F8 F7 F6 F5 F4 F3 F2 F1 F0 EF EE ED EC EB EA E9 E8 E7 E6 E5 E4 E3 E2 E1 E0 DF DE DD DC DB
/tb/X FF FF F5 EB E1 D7 CD C3 B9 AF A5 9B 91 87 7D 73 69 5F 55 4B 41 37 2D 23 19 0F 05 FB F1 E7 DD D3 C9 BF B5 AB A1 97
/tb/ALUFN 08 08 09 0A 0B 0C 08 09 0F 10 11 12 10 11 12 19 1A 1D 1F 1B
/tb/ALUout FE FE F3 12 1B 29 33 FA F9 F6 F5 90 85 6C 75 00 80 C0 1D 76 00 48 D0 01 07 00 E7 FF C1 C0 60 6B 89 82 4C
/tb/Nflag 1
/tb/Cflag 1
/tb/Zflag 0
/tb/Vflag 0